

Architecture alternative

Solveur C + base de donnees

Knight's Labyrinth - CIR1 2025-2026

Abandon du solveur JavaScript (Web Worker)
Pre-calcul des solutions via programme C
Lecture des niveaux depuis MySQL (table niveau)

Document technique - Mai 2026

1. Contexte et objectif

Le projet Knight's Labyrinth propose aujourd'hui un panneau OPTIMAL PATH alimenté par trois mécanismes en cascade : (1) cache serveur (colonnes `solution_cache` et `solution_safe` en base), (2) Web Worker JavaScript (`solver-worker.js`) avec BFS optimal et contrainte fantomes, (3) repli gems-only si le BFS sur ne trouve pas de chemin sûr en temps imparti.

L'objectif de cette étude est de définir une architecture où le calcul des solutions est exclusivement réalisé en C, les niveaux étant lus depuis la base MySQL, puis écrits dans `solution_cache`. Le navigateur ne ferait plus que consommer le cache via `get_solution.php` - suppression du Worker JS.

1.1 Etat actuel du depot

- Solveur C (dossier `solver/`) : `kl_solve()` gems-only, `kl_solve_safe()` évite R/G/B, jaune ignore.
- Solveur JS (`solver-worker.js` + `game.js solvePath`) : BFS complet avec fantôme jaune dans l'état.
- BDD : table `niveau` (`id`, `map TEXT`, `solution_cache`, `solution_safe`), 10 niveaux pré-remplis.
- API : `get_solution.php` retourne la séquence si `cached=true`.

1.2 Ecart fonctionnel à combler

Le solveur C en mode `--safe-path` ne modélise pas le fantôme jaune (dépend de la direction du joueur). Le Worker JS, lui, l'inclut dans l'espace d'états BFS. Avant de remplacer le JS, il faudra porter cette logique en C (voir section 5) ou accepter un écart documenté entre `solution_safe` en BDD et comportement réel en jeu.

2. Voies d'architecture possibles

Cinq approches realistes pour recuperer les niveaux en BDD et les resoudre en C. Toutes supposent un binaire solver compile (gcc -O2) et les colonnes solution_cache / solution_safe deja presentes.

2.1 Voie A - Batch CLI orchestre par PHP (recommandee)

Un script PHP (cli/solve_all_levels.php) se connecte en PDO, SELECT id, map FROM niveau, ecrit chaque carte dans un fichier temporaire level_{id}.txt, appelle exec() ou proc_open() sur ./solver --safe-path --moves-only, capture stdout (sequence UDLR), puis UPDATE niveau SET solution_cache=?, solution_safe=1 WHERE id=?.

- Avantages : simple, reutilise le code C existant, pas de lib MySQL en C, facile a deboguer.
- Inconvenients : I/O disque temporaires ; latence exec() negligeable en batch.
- Usage : cron nocturne, bouton admin apres creation d'un niveau, migration initiale.

2.2 Voie B - Programme C autonome avec connecteur MySQL

Etendre solver/ avec un mode batch : ./solver --db-host=localhost --solve-all. Liaison libmysqlclient ou MariaDB Connector/C : requete directe, parsing map en memoire, UPDATE sans fichier intermediaire.

- Avantages : performance I/O maximale, un seul processus, deployable sans PHP.
- Inconvenients : dependance native, gestion credentials, compilation plus complexe (LDFLAGS).
- Pertinent si equipe veut un outil DevOps pur C hors stack MAMP.

2.3 Voie C - Calcul a la demande via API PHP + exec()

Lors de la creation ou modification d'un niveau (formulaire admin), PHP appelle le binaire C de facon synchrone et renvoie la solution ou une erreur HTTP 503 si timeout.

- Avantages : toujours a jour sans cron ; pas de Worker JS cote client.
- Inconvenients : risque de blocage du thread Apache (30s+ sur grands niveaux) ; il faut set_time_limit, file d'attente ou job async pour la prod.
- A eviter pour le panneau OPTIMAL PATH en jeu ; reserve a l'admin.

2.4 Voie D - Service daemon / file de jobs

File Redis ou table jobs (level_id, status). Un worker longue duree (C ou PHP) consomme les taches, ecrit solution_cache. L'API get_solution.php renvoie cached=false tant que status=pending - le client affiche En cours de calcul sans Worker JS.

- Avantages : scalable, pas de timeout HTTP, UX propre pour niveaux utilisateur.
- Inconvenients : infrastructure supplementaire ; sur-projet pour 10 niveaux fixes.
- Utile si les joueurs peuvent publier des niveaux custom a grande echelle.

2.5 Voie E - WebAssembly (compromis, hors scope abandon JS)

Compiler le solveur C en WASM et l'appeler depuis le navigateur conserve une logique unique tout en restant cote client. Ce n'est pas l'option demandee (abandon du solveur JS) mais les benchmarks montrent un gain typique de 2 a 5x vs JS pur pour le BFS. Mentionne ici a titre comparatif uniquement.

2.6 Synthese comparative des voies

Voie	Complexite	Perf batch	Prod web	Recommandation
------	------------	------------	----------	----------------

A PHP+exec	Faible	Bonne	Admin OK	OUI - MVP
B C+MySQL	Moyenne	Excellente	Admin OK	OUI - v2
C exec sync	Faible	N/A	Risquee	Admin seulement
D file jobs	Elevee	Excellente	Ideale	Si niveaux user
E WASM	Elevee	Client	Evite PHP	Non (hors cahier)

3. Flux cible apres migration

Chargement du jeu (game.php) : le client appelle uniquement get_solution.php. Si solution_cache est renseigné, affichage immediat. Sinon message Solution non disponible - contactez l'admin (plus de fallback Worker).

Pipeline batch (une fois par niveau nouveau ou modifie) :

- 1. SELECT id, map FROM niveau WHERE solution_cache IS NULL OR solution_cache = "";
- 2. Pour chaque id : ecrire map -> /tmp/kl_level_{id}.txt
- 3. Executer : solver /tmp/kl_level_{id}.txt --safe-path --moves-only
- 4. Si code retour 0 et sortie != UNSOLVABLE : UPDATE solution_cache, solution_safe=1
- 5. Sinon : solution_safe=0, log erreur, eventuellement retry gems-only (--sans --safe-path)
- 6. Supprimer fichiers temporaires

HINT FROM HERE : deux options. (a) Pre-calcul impossible pour tous les etats intermediaires - desactiver ou reserver a un batch lourd. (b) Etendre le batch pour stocker des sous-solutions (hors scope MVP).
Solution pragmatique : garder un mini-solveur C invoque via API admin uniquement, ou accepter hint desactive en prod.

4. Le solveur C est-il plus performant que le solveur JS ?

Reponse courte : OUI, de maniere significative pour un BFS a espace d'etats large, surtout en batch hors navigateur. En revanche, pour l'affichage en jeu, seul le cache compte une fois la migration faite - la perf runtime du solveur ne se voit plus.

4.1 Facteurs techniques

- Compilation native (-O2/-O3) : pas d'interpretation JIT, acces memoire direct.
- Structures compactes : uint64_t pour les cle d'etat vs chaines JS dans Map.
- Pas de GC : le BFS alloue des tableaux fixes (8M noeuds) sans pauses collecteur.
- Worker JS : deja hors thread UI, mais reste V8 avec overhead objets et Map.

4.2 Ordres de grandeur (sources publiques)

Benchmarks generaux (programming-language-benchmarks, comparaisons C vs JavaScript) : le C est typiquement 5x a 50x plus rapide sur boucles numeriques intensives selon l'algorithme et la qualite du code JS.

Benchmark BFS dedie (projet Sable/bfs-benchmark, graphes petits/moyens) : C++ natif environ 10x plus rapide que JS Node sur petites instances, 2,5x sur instances moyennes. Le navigateur (Chrome) est encore plus lent que Node sur les memes tests.

Pour Knight's Labyrinth : un etat safe inclut jusqu'a 30 gemmes (2^{30} masques), tour, position jaune - l'espace explose. Le Worker limite a 8M noeuds / 30s ; le C utilise la meme limite (BFS_MAX_NODES) mais parcourt plus de noeuds par seconde, donc plus de chances de trouver une solution safe avant timeout.

4.3 Tableau comparatif applicatif au projet

Critere	Solveur C (batch)	Worker JS (actuel)
Debit BFS	Eleve (natif)	Moyen (V8)
Memoire etat	Cle 64 bits	Cle string Map
Fantome jaune	A implementer	Implemente
Usage en jeu	Via cache BDD	Direct ou cache
Blocage UI	Aucun (hors browser)	Aucun (Worker)
Deploiement MAMP	exec + binaire	Aucun binaire

4.4 Conclusion performance

Pour pre-calculer les 10 niveaux (ou des centaines en batch), le C est le bon choix : temps total reduit, logs reproductibles, pas de limite navigateur. Pour l'experience joueur apres migration, les deux approches sont equivalentes si tout est en cache. Le gain C se manifeste surtout a la creation de contenu et pour des niveaux difficiles ou le JS timeout souvent.

5. Evolutions du solveur C avant remplacement du JS

- Porter stepYellow() et l'etat (yr, yc, yd) dans kl_solve_safe (aligner sur solver-worker.js).
- Mode --moves-only deja present ; ajouter --json pour sortie machine (id, moves, safe, ms).
- Mode --stdin ou --map-string pour eviter fichiers temporaires depuis PHP.
- Codes retour explicites : 0=solvable, 1=unsolvable, 2=erreur parse, 3=limite memoire.
- Option --timeout N secondes (alarm) pour batch robuste.

5.1 Plan de migration (phases)

- Phase 1 : Completer kl_solve_safe (jaune) + tests sur levels/ et maps BDD.
- Phase 2 : Script PHP cli/solve_all_levels.php + Makefile install target.
- Phase 3 : Re-generer solution_cache pour les 10 niveaux ; verifier en jeu.
- Phase 4 : Retirer solver-worker.js, simplify game.js (cache only).
- Phase 5 : Documenter procedure admin pour nouveaux niveaux.

5.2 Esquisse script PHP (voie A)

```
<?php
// cli/solve_all_levels.php
$cfg = require __DIR__.'../web/includes/config.php';
$pdo = new PDO(...);
$solver = realpath(__DIR__.'../solver/solver');
foreach ($pdo->query('SELECT id, map FROM niveau') as $row) {
    $tmp = sys_get_temp_dir().'/kl_'.$row['id'].'.txt';
    file_put_contents($tmp, $row['map']);
    $out = shell_exec(escapeshellarg($solver).' '.escapeshellarg($tmp).' --safe-path --moves-only 2>&1');
    // UPDATE si solvable...
}
```

5.3 Securite exec()

Utiliser escapeshellarg(), chemin absolu vers le binaire, desactiver l'appel depuis le web public (CLI ou endpoint admin protege par session + CSRF). Ne jamais passer du contenu map non valide directement en argument shell (toujours via fichier temp).

6. Recommandation finale

Adopter la voie A (batch PHP + binaire C) comme MVP, puis évoluer vers la voie B si l'équipe souhaite un outil C autonome sans PHP. Abandonner `solver-worker.js` une fois tous les `niveau.solution_cache` valides. Compléter le solveur C pour le fantôme jaune afin que `solution_safe=1` soit fiable face au moteur de jeu.

Le solveur C est objectivement plus performant que le JS pour ce problème ; l'intérêt principal n'est pas d'accélérer l'affichage en partie (déjà instantané via cache) mais de fiabiliser le pré-calcul, supprimer la complexité navigateur, et traiter des niveaux plus difficiles hors ligne.

6.1 References

- Depot projet : `solver/solver.c`, `web/js/solver-worker.js`, `sql/schema.sql`
- Benchmarks langages : `programming-language-benchmarks.vercel.app` (JS vs C)
- BFS benchmark : `github.com/Sable/bfs-benchmark`
- README projet section 7 (solveur trois modes)